Modalità di svolgimento dell'esercitazione

Questa volta viene richiesto di scrivere funzioni per risolvere vari problemi, e viene fornito anche uno strumento che mette alla prova il comportamento delle funzioni che avete scritto.

Create una cartella, nella quale saranno creati sia i file contenenti le funzioni che avete scritto (una per ciascun file), sia i file di test (forniti dal docente).
I nomi di questi file devono essere esattamente i seguenti:

- `labEserc6_1.py`, `labEserc6_2.py`, `labEserc6_3.py`, `labEserc6_4.py`, `labEserc6_5.py`, per i cinque files contenenti le funzioni scritte da voi,
- `test_labEserc6_1.py`, `test_labEserc6_2.py`, `test_labEserc6_3.py`, `test_labEserc6_4.py`, `test_labEserc6_5.py` per i file di test (forniti dal docente)

Per ciascuno dei 5 problemi elencati nel seguito dovrete scrivere un file contenente la soluzione richiesta, in cui **il nome della funzione deve essere esattamente quello indicato nel testo seguente**.

Una volta salvato il file nella cartella, e salvato anche il corrispondente file di test fornito dal docente, dovete eseguire il comando

```
python3 test_labEserc6_X.py
```

dove `X` è il numero da 1 a 5 corrispondente all'esercizio che volete verificare.

Il risultato del test è una schermata (o più schermate) nella quale si mostra:

- se è stato trovato il file con il nome corretto,
- se il file contiene la funzione con il nome corretto,
- se chiamate alla funzione con diversi valori dei parametri terminano restituendo risultati corretti.

Per ogni funzione scritta vengono eseguite chiamate con diversi valori dei parametri. L'esito dei test viene riportato con il carattere

- **E** se la chiamata non può essere eseguita,
- **F** se la funzione non restituisce il risultato corretto,
- **.** se la funzione restituisce il risultato corretto.

Semplici algoritmi su liste

Primo esercizio: `somma_liste`

Scrivere una funzione `somma_liste(a, b)` che:

- riceve come parametri due liste `a`, `b` della stessa lunghezza, in cui ciascun elemento è un numero `int`;
- solleva una eccezione `TypeError` se i parametri ricevuti non sono liste, o se gli elementi delle liste non sono tutti numeri interi;

- solleva una eccezione di tipo `ValueError` se le liste non hanno la stessa lunghezza;
- crea e restituisce una lista, della stessa lunghezza di quelle ricevute come parametri, in cui l'elemento i-esimo della lista è uguale alla somma dell'elemento i-esimo di `a` e dell'elemento i-esimo di `b`

Il file di test da memorizzare nella propria cartella per questo esercizio è
test_labEserc6_1.py
La soluzione deve essere prodotta in un file di nome `labEserc6_1.py`

Secondo esercizio: `prodotto_scalare`

Scrivere una funzione `prodotto_scalare(a, b)` che:

- riceve come parametri due liste non vuote `a`, `b` della stessa lunghezza, in cui ciascun elemento è un numero (`int` oppure `float`);
- solleva una eccezione `TypeError` se i parametri ricevuti non sono liste, o gli elementi delle liste non sono tutti numeri;
- solleva una eccezione di tipo `ValueError` se le liste non hanno la stessa lunghezza oppure sono liste vuote;
- restituisce il prodotto scalare delle due liste. Il prodotto scalare tra due vettori NON È UN VETTORE, bensì un numero: il prodotto scalare è la somma dei prodotti degli elementi omologhi:
  ad esempio, il prodotto scalare tra i vettori (1, 4, 3) e (5, 5, 1) è 1*5 + 4*5 + 3*1 = 28.

Il file di test da memorizzare nella propria cartella per questo esercizio è
test_labEserc6_2.py
La soluzione deve essere prodotta in un file di nome `labEserc6_2.py`

Terzo esercizio: `separa_elementi`

Scrivere una funzione `separa_elementi(valori)` che:

- riceve come parametro una lista `valori`, in cui ciascun elemento è un numero (`int` oppure `float`);
- solleva una eccezione `TypeError` se il parametro ricevuto non è una lista, o se gli elementi della lista non sono tutti numeri;
- restituisce due liste (notare che è ammessa l'istruzione `return lista1, lista2`). La prima lista deve contenere tutti gli elementi di `valori` che sono maggiori della media degli elementi di `valori`, mentre la seconda lista deve contenere tutti gli elementi di `valori` che sono minori o uguali alla media degli elementi di `valori`.
  Ad esempio, se la lista contenesse i valori [0.96, 0.2, 0.4, 0.1, 0.55, 0.03, 0.88], poiché la media degli elementi è 0.4457, si dovrebbero restituire le liste [0.96, 0.55, 0.88] e [0.2, 0.4, 0.1, 0.03]
- nel caso di una lista vuota, la funzione deve restituire una coppia di liste vuote (non ha senso calcolare la media dei valori)

Il file di test da memorizzare nella propria cartella per questo esercizio è
test_labEserc6_3.py
La soluzione deve essere prodotta in un file di nome `labEserc6_3.py`

Quarto esercizio: `intersezione`

Scrivere una funzione `intersezione(a, b)` che:

- riceve come parametri due liste `a`, `b`;
- solleva una eccezione `TypeError` se i parametri ricevuti non sono liste;
- restituisce una lista **ordinata** che rappresenta l'intersezione tra le due liste, cioè gli elementi che appartengono sia alla lista `a` che alla lista `b`. Si può assumere che `a` non contenga elementi duplicati e che `b` non contenga elementi duplicati.
  Per ordinare il risultato, prima di restituirlo, basta chiamare il metodo `sort`, ad esempio con `risultato.sort()`

Il file di test da memorizzare nella propria cartella per questo esercizio è `test_intersezione.py`
La soluzione deve essere prodotta in un file di nome `intersezione.py`

Quinto esercizio: `unione`

Scrivere una funzione `unione(a, b)` che:

- riceve come parametri due liste `a`, `b`;
- solleva una eccezione `TypeError` se i parametri ricevuti non sono liste;
- restituisce una lista **ordinata** che rappresenta l'unione tra le due liste, cioè gli elementi che appartengono alla lista `a` e/o alla lista `b`. Si può assumere che `a` non contenga elementi duplicati e che `b` non contenga elementi duplicati.
  Per ordinare il risultato, prima di restituirlo, basta chiamare il metodo `sort`, ad esempio con `risultato.sort()`

Il file di test da memorizzare nella propria cartella per questo esercizio è `test_unione.py`
La soluzione deve essere prodotta in un file di nome `unione.py`